

Final Project

Decentralised Lending Service

REPORT

Peer2Peer Systems and Blockchains

Francesco Polperio 635406

Federico Lombardi 715426

Assumption

The most significant choice concerns the semantics of the compensation pool: we interpret a compensation as a restoration of the contributor's disposable value on the funding pool, rather than as a direct ETH transfer to the contributor's wallet.

The LendingPool contract implements a custom reentrancy guard via the state variable `_reentrancyStatus` (with values 1 = free, 2 = entered) and a local `nonReentrant` modifier, rather than inheriting from OpenZeppelin's `ReentrancyGuardUpgradeable` (to have smaller bytecode).

Project structure

The project is organized into three main layers:

Smart contracts (contracts/): On-chain logic: `LendingPool`, `LoanContract`, `BitcoinOracle`, and `LocalProxy`. A **vulnerable/** subfolder hosts the reentrancy-exposed variant and the attacker contract; **mocks/** contains fake oracle and a V2 implementation used to validate upgradability.

Off-chain Python services (oracle/, scripts/): `oracle_service.py`, `InitialSetup.py` bootstraps the private chain, `DemoOperations.py`, `YesMan.py` contributor that always votes approve, `GasMeasurement.py`.

Tests (test/): Hardhat test suite.

```
project_p2p/
├── contracts/
│   ├── LendingPool.sol           # Solidity smart contracts
│   ├── LoanContract.sol         # lending pool, UUPS upgradeable + governance
│   ├── BitcoinOracle.sol        # single loan logic
│   ├── LocalProxy.sol           # BTC price oracle
│   └── mocks/                   # ERC1967 proxy for upgrades
│       ├── LendingPoolV2.sol    # mock contracts for tests
│       ├── MockBitcoinOracle.sol # V2 version (upgrade test)
│       └── vulnerable/          # fake oracle
│           ├── LendingPoolVulnerable.sol # intentionally flawed versions for attack demos
│           └── ReentrancyAttacker.sol
├── test/                         # Hardhat tests (JS, Mocha/Chai) - 9 suites
│   ├── LendingPool.test.js
│   ├── Loan.test.js
│   ├── Oracle.test.js
│   ├── OracleFee.test.js
│   ├── Proposal.test.js
│   ├── Reentrancy.test.js
│   ├── Resolve.test.js
│   ├── Termination.test.js
│   └── Upgradability.test.js
├── scripts/                      # Automation
│   ├── InitialSetup.py          # deploy + initial setup
│   ├── DemoOperations.py        # demo operations
│   ├── GasMeasurement.py        # gas measurement -> data/gas_report.csv
│   ├── YesMan.py                # proposal auto-voter
│   └── start_chain.sh           # launches local geth chain
├── oracle/                      # Off-chain oracle service
│   └── oracle_service.py        # feeds BitcoinOracle
├── frontend/                    # React + Vite + Tailwind dApp
├── docs/                        # Documentation
│   └── v011_ProjectP2PBC_LAB2526.pdf # project assignment
├── README.md                   # Project README
├── hardhat.config.js           # Hardhat config
├── package.json                # Solidity deps (OpenZeppelin) + tests
├── requirements.txt             # Python deps
└── project2526genesis.json      # geth blockchain genesis
```

Figure 1: Project directory structure

Implementation choices: On-Chain Components

LendingPool

The `LendingPool` is the core contract of the service and the source of truth for the state of the pool. It is the custodian of all ETH deposited by contributors and the entry point through which every user-facing operation flows. On top of pure custody, it embeds the governance engine of the service, acts as a factory for per-loan contracts, and bookkeeps every value movement between contributors, applicants, and active loans.

A second pool of value lives inside the same contract, the compensation pool, which collects the collateral fraction of past interest and covers failed loans, removing the need for additional value transfers between contracts.

State and Accounting

Each contributor's total deposit is tracked in `deposits[]`, the portion currently locked in active loans in `lockedValue[]`; the difference, computed on demand by `disposableValue`, is the spendable balance used for withdrawals and vote weighting.

The aggregates `totalFundingPool`, `totalLocked` and `compensationPool` mirror these per-contributor quantities at the global level, so `totalDisposable` is evaluated without iterating the contributor list. A separate `_contributorList`, guarded by an anti-duplicate map, is appended on first deposit, it is iterated only at resolution time and zero-disposable entries are skipped.

Deposit and Withdraw

The `deposit` function accepts any value greater than or equal to `MIN_DEPOSIT = 100,000 wei`, registers the sender in the contributor list on first call, and updates both `deposits[]` and `totalFundingPool`.

The `withdraw` function enforces a strict CEI ordering: the disposable check is performed first, then state is decremented, and only at the end the ETH transfer is performed via `call`. The whole function sits behind the local reentrancy mutex.

Proposal Lifecycle

`submitProposal` stores the applicant, the requested amount, the interest rate, the duration, the keccak hash of the Bitcoin address, and the current block number. The proposal struct nests two mappings (`hasVoted` and `voteApprove`), so `_proposals` is kept internal and the public surface is exposed through explicit getters (`getProposal`, `hasVotedOn`, `getVoteApprove`).

Only approve voters are pushed into the iterable `approveVoters[]` array, while reject voters are tracked only through the `hasVoted` flag, so the resolution loop walks only the affirmative side.

Each contributor's share is computed as:

$$\lfloor \text{amount} \times \frac{\text{disp}_i}{\text{totalDisp}} \rfloor$$

Contributors are then sorted by share descending, ties broken by ascending address.

Upgradability

`LendingPool` is deployed behind an ERC-1967 proxy and follows the UUPS pattern: only the proxy address is stable over time, while the implementation can be replaced by the owner.

The `_authorizeUpgrade` hook is gated by `onlyOwner`, and the implementation constructor calls `_disableInitializers` so the bare implementation cannot be initialised outside the proxy. The storage layout is strictly append-only, verified in the test suite against `LendingPoolV2`: the mock appends one new slot and a new two functions on top of v1 and exposes a `reinitializer(2)` hook that ensures the v2 migration logic can run at most once.

LoanContract

The `LoanContract` is the per-loan component of the service: one instance is deployed by the `LendingPool` for each approved proposal and exists for the entire life of that single loan. Its role is to isolate the state and the logic of one loan from all the others. It is the natural interlocutor of the applicant for repayments and of the contributors for compensation requests.

Disbursement

Its constructor records the contributor list, the per-contributor initial locks, the expected interest tranches (proportional to the initial locks), and immediately forwards the loaned amount to the applicant in the same transaction.

The snapshot of each contributor's initial locked share is frozen at this point and becomes the reference for all subsequent proportional computations (interest distribution, base repayments, compensation entitlement).

Partial Repayment

`partialRepay` splits the incoming value between base and interest. The base component, capped at the remaining loan amount, is distributed by walking the pre-sorted contributors and refunding each one up to their residual capacity:

$$\text{initialLocked} - \text{unlockedSoFar} - \text{compRecovered}$$

If a contributor has already drawn from the compensation pool, the helper `_splitBaseForfeit` diverts a proportional slice of their share back to the compensation pool, implementing the spec rule that requesting a compensation forfeits the corresponding applicant repayment.

Any value exceeding loan plus interest is credited to the compensation pool.

Compensation

`requestCompensation` can be called by any contributor at any time while the loan is not **Successful**. On the first call after `expiryBlock` the status transitions to **Failed**, `increaseCollateral` fires on the pool, and the remaining expected interest is set to zero, so any later repayment goes entirely to the compensation pool. The compensation amount is the minimum between owed value and the value stored in the compensation pool. The actual movement, on the pool side, is a transfer from `lockedValue` back to disposable: the compensation pool decreases, the contributor's locked value decreases, the deposit is left unchanged.

Termination

`terminate` is callable only when the loan is **Successful** or when every contributor on a **Failed** loan has been either fully repaid or fully compensated. It sweeps any residual ETH on the loan contract to the compensation pool, deregisters the loan through `markLoanClosed`, and sets an idempotent `terminated` flag so the call cannot run twice.

BitcoinOracle

The `BitcoinOracle` gives the `LendingPool` a queryable view of the available balance of a Bitcoin address.

Trust is concentrated on a single privileged role, the `operator`, established at deployment and the only address authorized to push new balance values into the contract storage.

Balances are stored under the key `keccak256(btcAddress)` as raw satoshi values. `requestUpdate` is paid (`msg.value ≥ MIN_ORACLE_FEE`) and emits the event consumed by the off-chain service; `update`, gated by `onlyOperator`, commits the fresh balance.

`MIN_ORACLE_FEE` is hardcoded, being the measured gas cost of the update on the private chain.

Auxiliary files

- `LocalERC1967Proxy` is a minimal wrapper around the OpenZeppelin proxy, recompiled locally.
- `LendingPoolV2` is a mock target used only to exercise the UUPS upgrade.

Implementation choices: Off-Chain Components

Oracle Service

The file `oracle/oracle_service.py` parses mainnet blocks from the local `blk*.dat` files, maintains a per-address satoshi UTXO map, listens for `UpdateRequested` events on the on-chain oracle contract, and submits the corresponding update transactions signed by the operator key.

Automation Scripts

`InitialSetup.py` creates EOAs, funds them from the prefunded miner account, deploys the production contracts behind the proxy and writes the resulting configuration files. `DemoOperations.py` runs a complete end-to-end scenario, printing the balances of the involved accounts and the relevant pool state after every meaningful step. `Gas-Measurement.py` produces the gas report. `YesMan.py` is the automated contributor required by the assignment: it watches `ProposalSubmitted` events and approves every new proposal as soon as it sees one.

Events

State transitions are exposed to off-chain consumers through a set of events.

- The pool emits `Deposited` and `Withdrawn` on funding-pool movements, the full proposal lifecycle through `ProposalSubmitted`, `ProposalVoted`, `ProposalApproved` and `ProposalRejected`, and `LoanRegistered`, `LoanDeregistered`, `CollateralPercentageChanged` on the loan registry.
- Each loan contract emits `LoanCreated` at construction, `Repayment` (with base, interest, excess to comp and residuals) on every `partialRepay`, `MarkedFailed` on the transition to **Failed**, `CompensationRequested(owed, paid)` on each compensation attempt, `LoanClosed` on settlement, and `LoanTerminated` on `terminate`.
- The oracle emits `UpdateRequested` and `BalanceUpdated`. `ProposalSubmitted` is consumed by the `YesMan` automated voter, while `UpdateRequested` and `BalanceUpdated` are the integration points of the off-chain oracle service.

Gas evaluation

The full output is written in `data/gas_report.csv`.

Results

Operation (scenario)	Gas	Cost (ETH)
<code>deposit</code> – new contributor	138,734	1.39e-4
<code>deposit</code> – existing contributor	40,064	4.01e-5
<code>withdraw</code> – partial	49,895	4.99e-5
<code>requestOracleUpdate</code> – via pool forward	40,951	4.10e-5
<code>update</code> – new address	45,703	4.57e-5
<code>update</code> – existing address	28,603	2.86e-5
<code>submitProposal</code> – valid (amount ≤ disp, btc ok)	184,376	1.84e-4
<code>vote</code> – approve	123,881	1.24e-4
<code>vote</code> – reject	59,653	5.97e-5
<code>resolveProposal</code> – Approved (N=2)	1,972,086	1.97e-3
<code>resolveProposal</code> – Approved (N=5)	2,344,914	2.34e-3
<code>resolveProposal</code> – Rejected (pool low)	63,139	6.31e-5
<code>resolveProposal</code> – Rejected (btc liquidity)	73,084	7.31e-5
<code>resolveProposal</code> – Rejected (weighted vote)	82,345	8.23e-5
<code>partialRepay</code> – mid (no overpay)	81,566	8.16e-5
<code>partialRepay</code> – close Successful	190,005	1.90e-4
<code>partialRepay</code> – overpay (extra interest)	192,831	1.93e-4
<code>requestCompensation</code> – first call (marks Failed)	85,212	8.52e-5
<code>requestCompensation</code> – subsequent (refill claim)	47,698	4.77e-5
<code>partialRepay</code> – on Failed loan (proportional split)	150,058	1.50e-4
<code>terminate</code> – Successful loan	27,696	2.77e-5
<code>upgradeToAndCall</code> – UUPS v2 swap	37,704	3.77e-5

Reentrancy modification and attack

Vulnerable variant

The vulnerable contract `contracts/vulnerable/LendingPoolVulnerable.sol` is a clone of the real `LendingPool` with one change: in the `withdraw` function the external ETH transfer is performed *before* the state update, and the local `nonReentrant` guard is removed. The CEI ordering is therefore reversed, and the disposable balance check still passes when the call re-enters the function, because `deposits[]` has not yet been decremented.

Attacker contract

The attacker contract `contracts/vulnerable/ReentrancyAttacker.sol` is a minimal helper holding: the address of the vulnerable pool, the owner, the recorded `attackAmount` and an attack counter.

The attacker first calls `deposit`, which records `attackAmount` and registers the stake on the vulnerable pool. The owner then calls `attack`, which invokes `withdraw(attackAmount)` on the pool. The pool transfers the ETH back to the attacker through a low-level `call`.

The attacker's `receive` function then re-enters `withdraw` with the same amount. Because the pool has not yet decremented its bookkeeping, the disposable check still succeeds, and the loop continues until either the maximum number of re-entries is reached or the pool runs out of balance. At the end of the cascade, the attacker has withdrawn the original stake plus the drained value, while the funds of other contributors become unreachable on the pool.

Malicious strategy

The resolution rule approves a proposal as soon as the weighted yes votes exceed half of the disposable pool, so a single contributor holding more than half of the disposable value can unilaterally approve any proposal, including their own.

Furthermore, the specification explicitly allows the same address to act as applicant and contributor on the same loan.

The problem's origin is the fact that contributors are refunded in descending order of initial locked value, with ties broken by ascending address, so whoever locked the largest share is reimbursed first.

Paramount aspect is the absence of any ownership proof or locking on the BTC collateral: the applicant can pledge any rich Bitcoin address, the oracle balance is a one-shot snapshot, and there is no clawback if the BTC is moved after the resolution.

Malicious walkthrough

The exploit is executed by an attacker, Alice, who controls more than half of the disposable pool and holds a BTC address with a balance sufficient to cover the loan. The attack vector begins when Alice triggers an oracle update for the hash of her BTC address and subsequently submits a proposal where she acts as both the applicant and the

majority funder. During the voting phase, she casts a positive vote. Because she controls the majority of the pool, her vote alone satisfies the weighted-majority check.

Once the voting period concludes, Alice requests the resolve of the proposal. All system checks pass because the proportional split recognizes her as the largest shareholder. Consequently, a fresh loan contract is deployed, and the funds are disbursed directly to her wallet. Immediately after receiving the funds, Alice moves the BTC away from the provided BTC address. Since the Ethereum side never re-reads the oracle after this point, the state remains unaffected.

Alice then repays partially, paying back only the portion of the capital that matches her own share, never reaching the honest contributors.

After the loan duration elapses, the contract is left with unpaid capital representing the honest contributors' share, along with unpaid interest. When the honest contributors eventually request a compensation, the loan is failed. Finally, Alice safely withdraws her now-unlocked ETH deposit if she wants.

Outcome

Alice walks away with her original deposit completely intact, supplemented by the exact share of capital contributed by the honest users, while her original BTC collateral remains untouched. The honest contributors suffer a total loss of their economic stake. The entire exploitation process requires only negligible gas and oracle fees from the attacker.

User manual

```
# Python and frontend dependencies (venv) - first time only
```

```
./scripts/install.sh
```

```
# Before startup: put .blk files in ./chaindata
```

```
# First terminal - Start the chain
```

```
./scripts/start_chain.sh
```

```
# Second terminal
```

```
source venv/bin/activate #if not already activated automatically use this command
```

```
npm hardhat compile
```

```
python3 scripts/InitialSetup.py #wait the end of the execution before proceeding
```

```
OPERATOR_PRIVATE_KEY=$(jq -r .oracle_operator.key data/accounts.json) \  
python3 oracle/oracle_service.py
```

```
# Third terminal (optional) - Frontend, dynamic visualization of DemoOperation
```

```
source venv/bin/activate #if not already activated automatically use this command
```

```
cd frontend && npm run dev
```

```
# open localhost address link
```

```
# Fourth terminal (optional) - YesMan
# order: geth node activated -> InitialSetup.py -> oracle activated

source venv/bin/activate    #if not already activated automatically use this command

python3 scripts/YesMan.py
```

```
# Fifth terminal

source venv/bin/activate    #if not already activated automatically use this command

python3 scripts/DemoOperations.py
```

```
# Terminal for tests

npx hardhat test
# needed the geth node activated

source venv/bin/activate    #if not already activated automatically use this command

python3 scripts/GasMeasurement.py
```
